SYDE 572 Course Project: Household item classification via ResNet50 transfer learning

Benjamin Jingwen Luo

3B Management Engineering (BASc), University of Waterloo, Ontario, Canada

Summary

This is the written report for the SYDE 572 (F23) course project on implementing a classifier for a dataset of common household items. It will overview the design choices and various approaches attempted throughout the project. The final model is a frozen ResNet50 with pre-trained weights which scored a testing accuracy of roughly 91% (±2%).

I. Data Processing

Household item dataset

The challenge is to classify a provided dataset of household items. The dataset is split into a labeled 'train' (N=5x22 data points) and unlabeled 'test' set (N=516 data points). A labeled dataset refers to a nested folder structure where the subdirectory's name is the class name. The class names are given as numbers which map to the strings in appendix 1.

All images are given as 1024x543 jpg files that have an item roughly centered in the middle of the same gray background. It is likely the photos were captured in the same environment as the lighting and zoom levels appear identical across images. Objects in the same class have roughly the same dimensions (length x width) but different orientations and colors.

Due to the small training set that consists of 5 images per class, it was necessary to manually label the provided test set by creating subdirectories with the desired class name and moving the test images accordingly. This directory is not used for directly training the model but may also bias the results as the model parameters are tuned with respect to the test set so it is no longer usable as a validation set.

Benchmark model and performance metric

The benchmark model is a ResNet50 with a testing accuracy of 80% (±1%). The performance metric is test accuracy, relative to a randomly sampled portion of the test set. Any implemented models are restricted to being equal or smaller in size, or having at most 25.6 million parameters.

Data loading

This report uses PyTorch as it provides powerful deep learning models. PyTorch offers a specialized image loading function called 'ImageFolder' which loads images into a data object and splits it into image data and label:

```
trainset = datasets.ImageFolder(root=train_dir, transform=train_transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)

testset = datasets.ImageFolder(test_dir, transform=test_transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size)
```

Figure 1: Data loading via the ImageFolder function

The above figure demonstrates how the train and test sets are loaded into their respective variables. The batch size represents how the data is further partitioned into 'batches' when they are passed into the model, and *shuffle=True* indicates the data is randomized upon loading.

Data augmentation

Prior to loading, the data is also augmented to better generalize to unseen data. The transformations are justified in figure 2.

```
train_transform = transforms.Compose([
    transforms.Resize(255),  # Resize to fit the model input size
    transforms.RandomHorizontalFlip(p=0.5),  # Flip because half the images are inverted
    transforms.RandomVerticalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])  # Fine-tuning
])

test_transform = transforms.Compose([
    transforms.Resize(255),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Figure 2: Data augmentation on the train and test sets

Originally there was also a center crop by 224 as all the images are centered but this was shown to

decrease model performance likely due to larger images (ex. keyboards) being cropped out of the frame and data loss. Cropping may also be unnecessary because all the images have the same background so the model will be able to more easily discern the subject from the noise.

Once loaded, the images are in the form 3x255x255 which indicates color channels (n=3) and image size (n=255x255).

II. Model Implementation

Model selection

Various models were attempted throughout the development process. The initial and most promising results stemmed from PyTorch's pretrained models. Namely (the best models are bolded):

- **ResNet50**, ResNet34
- **MobileNet v2**, MobileNet v3 (large/small)
- VGG19

The 2nd set of experiments were conducted with simpler ML tools provided by scikit-learn. Input vectors had to be preprocessed into dim=2 vectors via reshaping and PCA which resulted in significant data loss. These are:

- GradientBoostingClassifier
- RandomForestClassifier
- KNeighborsClassifier
- ExtraTreesClassifier
- SVC
- LinearDiscriminantAnalysis

The final experimental model is a simple rule-based model that involves manually measuring the average lengths and widths of each image, as well as other potential discriminators such as color, shape, orientation, and heterogeneity. However, this approach was quickly abandoned due to certain classes having very similar features (ex. book and cereal) which would make a rule-based system highly complicated.

In general, models were selected based on their popularity in the classification problem space, subject to having equal or less parameters than the benchmark ResNet50. Model testing and elimination are discussed in the 'Model evaluation' section.

Training and testing

Pretrained PyTorch models were imported and frozen, then had its final layer dimensions changed to 22 to match the number of household item classes. An example is shown in figure 3:

```
import torchvision.models as models

model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

for param in model.parameters():
  param.requires_grad = False

model.fc = nn.Linear(model.fc.in_features, 22, device=device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)
```

Figure 3: Sample model loading for ResNet50

The loss function is cross entropy loss which is the standard for multi-class classification problems. The optimizer is ADAM which showed to have more consistent results compared to SGD with the standard weight decay of 5e-4 and momentum of 0.9.

The training and testing code is copied and modified from the transfer learning tutorial code and are shown in appendices 2-4.

Hyperparameter optimization

Grid search was manually performed with respect to the hyperparameters of each model. Figure 4 shows an example parameter search space for ResNet50 with ADAM optimizer:

```
grid_params = {
    'max_epochs': [8, 10, 12, 14, 16, 18, 20], # Must be kept small for transfer learning
    'lr': [0.005, 0.01, 0.015],                # Optimal lr range for ADAM optimizer
    'batch_size': [8, 14, 16, 24, 36, 48, 64]  # Trying different values (for fine-tuning)
}
```

Figure 4: ResNet50 and ADAM hyperparameters

The range of each parameter is experimentally explored by a first-pass before exploiting near combinations with higher accuracies. Generally, ADAM benefitted from a learning rate between 0.005 and 0.015 and epoch count around 14. Batch size had inconsistent effects and was eventually kept at 16.

Exploration was also conducted by switching between ADAM and SGD optimizers as well as switching the pretrained weight configurations. For example, ResNet offers weights trained on ImageNet v1 and ImageNet v2, of which the latter yielded better results.

Roughly 600 parameter combinations were tested across all models and recorded in the attached files (Grid Search.txt; Grid Search (no crop).txt). The grid search code can be found in appendix 5 and sample output is shown below in figure 5:

```
ResNet50
Accuracy: 0.8955512642860413, epoch: 12, lr: 0.003, batch size: 16
Accuracy: 0.8936170339584351, epoch: 12, lr: 0.005, batch size: 16
Accuracy: 0.8800773620605469, epoch: 12, lr: 0.007, batch size: 16
Accuracy: 0.8955512642860413, epoch: 12, lr: 0.009, batch size: 16
Accuracy: 0.8781431317329407, epoch: 12, lr: 0.015, batch size: 16
Accuracy: 0.9071566462516785, epoch: 14, lr: 0.003, batch size: 16
Accuracy: 0.8955512642860413, epoch: 14, lr: 0.005, batch size: 16
```

Figure 5: Grid search output for ResNet 50 with ADAM optimizer for 7 trials

Simpler models from scikit-learn were tested by manually varying the parameters to determine whether further testing would be productive, in which case grid search would be conducted.

Visualizing outputs

A testing function was implemented to randomly select and output an image from the testing set, then returning the predicted and actual labels (Figure 6). The purpose of this is to visualize the image augmentations and where the model underperforms. The associated code is shown in appendix 6.

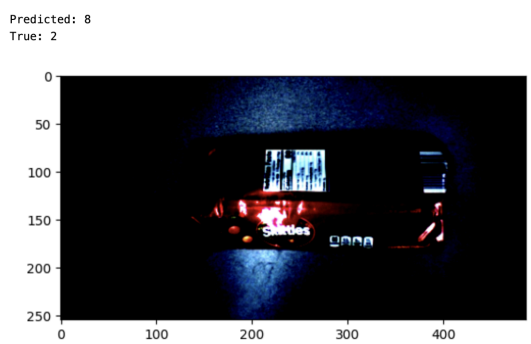

Figure 6: Randomly rendering an image from the test set and returning the actual v. predicted labels

Outputting results

The required output format is a CSV file with columns 'id' (the name of the test photo) and 'category' (the predicted label). A variant of the ImageFolder was created to preserve the image name when loading the data, then the unlabelled test set was loaded and classified [appendix 7]. Results were stored in a Pandas dataframe then output as a CSV.

Model evaluation

The 2 most promising models were the ResNet50 and MobileNet v2 which eventually scored accuracies between 88%–91%. Other pre-trained models scored around 70%–80% and scikit-learn classifiers roughly ~10%-40%.

For both ResNet50 and MobileNet v2, the optimizer was ADAM and the selected hyperparameters are:

- batch_size = 64
- lr = 0.003
- max_epochs = 18

12 replications were performed with each setup and results were consistent between the 88%–91% range with low standard deviation due to the small learning rate.

The scikit-learn models likely underperformed due to the PCA which reduced the vector size to 317 (99% of variance) down from 150528 (3x224x224). This was necessary to reduce runtime but the data loss was too significant to produce higher accuracies. Various PCA components and model hyperparameters were selected but the vast majority resulted in accuracies between ~10%–40%. The one exception is the support vector machine (SVM) with a linear kernel which had an accuracy of 84%. Another cause of the poor performance is the lack of training data as there were only 5 labels in each class. Certain classes may be undersampled after the train and test data split and k-fold validation split and therefore under-represented which leads to poor performance.

III. Appendix

Appendix 1: Class name mapping

0 = pen	11 = lipstick
1 = book	12 = lotion
2 = chocolates	13 = mouse
3 = cereal	14 = cup
4 = small comb	15 = plate
5 = fork	16 = shampoo
6 = glass	17 = lotion box
7 = big comb	18 = spatula
8 = hair tie	19 = spoon
9 = keyboard	20 = toothbrush
10 = knife	21 = toothpaste

Appendix 2: Training function from the transfer learning tutorial

```
def train(model,trainloader,optimizer,criterion):
  model.train()

  total_loss = []
  for data in trainloader:
    images, labels = data
    images = images.to(device)
    labels = labels.to(device)
    optimizer.zero_grad()

    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    total_loss.append(loss.item())
    optimizer.step()
  return np.average(total_loss)
```

Appendix 3: Testing function from the transfer learning tutorial

```
def eval_train(model, testloader, testset):
  model.eval()
  correct = 0.0

  for data in testloader:
    images, labels = data
    images = images.to(device)
    labels = labels.to(device)

    outputs = model(images)
    _,preds = outputs.max(1)
    correct += preds.eq(labels).sum()

  return correct.float()/len(testset)
```

Appendix 4: Code for training and evaluating the model

```
for epoch in range(max_epochs):
  train(model, trainloader, optimizer, criterion)

accu = eval_train(model, testloader, testset)
print('Accuracy: ', accu.item())
```

Appendix 5: Grid search code for ResNet50 with ImageNet v2 and ADAM optimizers

```
grid_params = {
    'max_epochs': [18],
    'lr': [0.003],
    'batch_size': [16]
}

for epochs in grid_params['max_epochs']:
    for lrs in grid_params['lr']:
        for batch_sizes in grid_params['batch_size']:
            trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_sizes, shuffle=True)
            testloader = torch.utils.data.DataLoader(testset, batch_size=batch_sizes)
            model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
            model.to(device=device)

            # Freeze model
            for param in model.parameters():
                param.requires_grad = False

            # Define final layer
            model.fc = nn.Linear(model.fc.in_features, 22, device=device)
            criterion = nn.CrossEntropyLoss()
            optimizer = optim.Adam(model.parameters(), lr=lrs)

            # Training
            for ep in range(epochs):
                train(model, trainloader, optimizer, criterion)

            # Test accuracy and output
            accu = eval_train(model, testloader, testset)
            print ('Accuracy: {}, epoch: {}, lr: {}, batch size: {}'.format(accu.item(), epochs, lrs, batch_sizes))
```

Appendix 6: Function to randomly render a test image and compute the true v. predicted labels

```
import matplotlib.pyplot as plt
import random

def rand_pred(model):
    model.eval()
    image, label = random.choice(testloader.dataset)
    mps_image = image.to(device)

    outputs = model(mps_image.unsqueeze(0))
    _,pred = outputs.max(1)

    print("Predicted:", classes[pred.item()])
    print("True:", classes[label])
    plt.imshow(image.permute(1, 2, 0))

rand_pred(model)
```

Appendix 7: Code for outputting predictions to a CSV file with the 2 columns ‘id’ and ‘category’

```
import pandas as pd

class ImageFolderWithPaths(datasets.ImageFolder):
    def __getitem__(self, index):
        img, label = super(ImageFolderWithPaths, self).__getitem__(index)
        path = self.imgs[index][0]
        return (img, label, path)

validationset = ImageFolderWithPaths(validation_dir, transform=test_transform)
validationloader = torch.utils.data.DataLoader(validationset)

def validation_pred(model):
    model.eval()
    predictions = []
    filenames = []
    with torch.no_grad():
        for images, _, paths in validationloader:
            images = images.to(device)
            output = model(images)
            _, predicted = torch.max(output, 1)
            predictions.extend(predicted.cpu().numpy())
            filenames.extend(paths)

    return predictions, filenames

predictions, filenames = validation_pred(model)
predictions = [classes[predict] for predict in predictions]
filenames = [int(filename.rsplit('/', 1)[1][:-4]) for filename in filenames]

# Create a DataFrame from the predictions
df = pd.DataFrame(predictions, columns=['category'])
df.insert(0, 'id', filenames)
df = df.sort_values(by=df.columns[0])

# Save the DataFrame to a CSV file
df.to_csv('20890448_luo.csv', index=False)
```